SIMATS ENGINEERING

(SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2

Architecture Design Assignment

Project Title: Intelligent AI-Based Airport Baggage Tracking and Management System

Student Name	RR Thiyagesh
Register Number	192572007
Course Code	CSA1017
Course Title	Software Engineering
CO Assessed	CO2 — Assessment Tool 2 (Architecture Design)
Weightage	20%	Total Marks: 25

CO2 Statement: Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills.

1. Introduction and Problem Context

Airports process millions of passenger bags every year, and the efficiency of baggage handling directly affects operational cost and passenger satisfaction. Conventional baggage-handling workflows rely on manual scans at fixed checkpoints, which provide only intermittent visibility into a bag's location and give airline staff little advance warning of misrouting or delay. The proposed system, the Intelligent AI-Based Airport Baggage Tracking and Management System, addresses this gap by combining RFID/IoT sensing, an event-driven backend, and machine-learning-based delay prediction to give passengers, ground staff, and airport operations teams continuous, real-time visibility into baggage status.

The system is expected to track baggage in real time, optimise routing decisions across sorting and transfer points, predict likely delays before they occur, notify passengers proactively, reduce misrouting incidents, and generate operational analytics that support continuous improvement of airport logistics.

2. System Requirements Analysis

2.1 System Objectives

- Provide real-time, end-to-end visibility of baggage movement from check-in to arrival.
- Optimise baggage routing across conveyor, sorting, and transfer infrastructure.
- Predict baggage delays proactively using AI/ML models trained on historical and live event data.
- Notify passengers automatically about baggage status and any predicted delay.
- Reduce baggage misrouting and the resulting operational cost and complaint volume.
- Generate operational analytics and reports for airport and airline management.

2.2 Functional Requirements

- The system shall register each bag with a unique RFID tag at check-in and associate it with the passenger's itinerary.
- The system shall capture RFID/IoT scan events at each checkpoint (check-in, sorting, loading, transfer, arrival) in real time.
- The system shall compute and update the optimal routing path for each bag as new scan events arrive.
- The system shall run a predictive model that estimates delay probability for each bag based on current handling status, flight connection time, and historical patterns.
- The system shall send automated notifications (push, SMS, or email) to passengers when a bag is scanned, delayed, or requires attention.
- The system shall provide airline and ground staff a dashboard to search, locate, and manually override the routing of a specific bag.
- The system shall generate periodic and on-demand operational reports summarising delay rates, misrouting incidents, and throughput.

2.3 Non-Functional Requirements

Attribute	Requirement
Scalability	Support peak loads of tens of thousands of concurrent scan events across multiple terminals withou
Performance	Process a scan event and reflect the updated status on the dashboard/app within 2-3 seconds.
Reliability	Guarantee at-least-once delivery of scan events between edge devices and backend services, with

Availability	Maintain 99.9% uptime for tracking and notification services during airport operating hours.
Security	Encrypt data in transit and at rest; enforce role-based access control for staff dashboards and passe
Maintainability	Allow individual services (e.g., prediction model, notification channel) to be updated or redeployed in
Interoperability	Integrate with existing airline departure control systems (DCS) and airport operational databases via

These quality attributes — particularly scalability (to absorb bursty scan traffic during peak flight schedules), performance (to keep tracking near real time), and reliability (to avoid losing scan events that determine baggage location) — are the primary drivers of the architectural style selected in Section 3.

3. Selection of Suitable Software Architecture

3.1 Selected Architectural Style

A **Hybrid Microservices + Event-Driven Architecture** is selected for this system. The system is decomposed into independently deployable microservices (baggage tracking, routing optimisation, AI delay prediction, notification, and analytics/reporting), which communicate asynchronously through a central event bus / message broker, and are exposed to clients through a single API gateway.

3.2 Justification

- **Real-time, high-volume event stream:** RFID/IoT scan events arrive continuously and in bursts. An event-driven backbone (publish-subscribe) decouples producers (sensors, tracking service) from consumers (prediction, notification, analytics), allowing each to scale and fail independently without blocking the scan pipeline.
- **Independent scalability:** Baggage tracking and notification services experience different, and highly variable, load profiles compared to analytics/reporting. Microservices allow each to be scaled horizontally on its own, avoiding the cost of scaling the entire monolith.
- **Isolation of the AI component:** The delay-prediction service can be updated, retrained, or redeployed independently of the rest of the system, which is important given that ML models require more frequent iteration than core tracking logic.
- **Fault isolation:** If the notification service or an external SMS/email gateway experiences an outage, baggage tracking and routing continue to function because they do not depend synchronously on notification delivery.
- **Integration with legacy/external systems:** Airports must integrate with existing airline Departure Control Systems (DCS) and airport operational databases. A gateway-fronted microservices layer provides clean API boundaries for this integration without exposing internal service topology.

Pure microservices without an event backbone would require many synchronous service-to-service calls to propagate a single scan event (tracking → routing → prediction → notification), increasing latency and coupling. A purely event-driven architecture without service boundaries, on the other hand, would make independent scaling, ownership, and deployment of the AI model difficult. The hybrid style combines the decoupling benefits of event-driven communication with the deployability and ownership benefits of microservices, and is therefore the most suitable choice for this problem.

4. Software Architecture Diagram

The diagram below illustrates the complete system structure across five layers: client/presentation, API gateway, application microservices, the event-bus integration layer, and the data layer, with external systems and edge devices at the base.

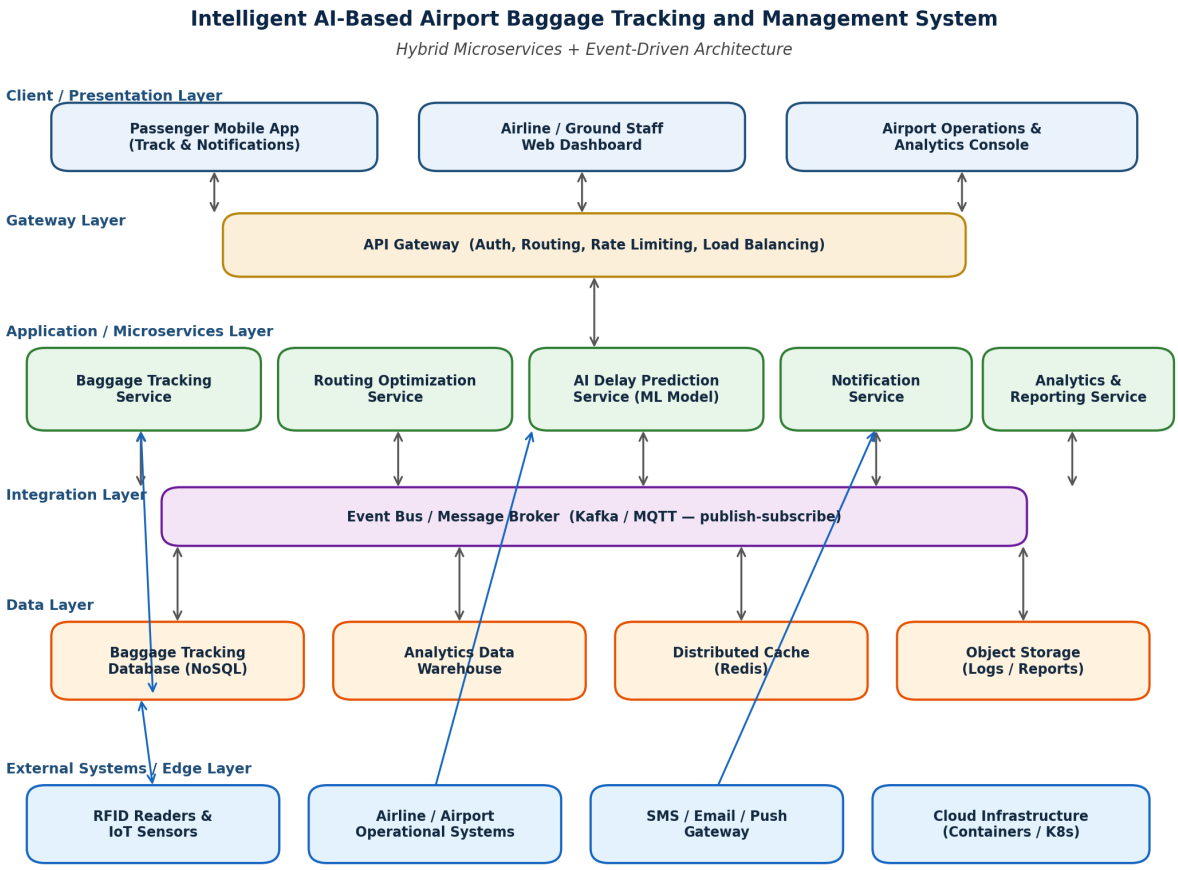

Figure 1: Overall System Architecture (Hybrid Microservices + Event-Driven).

5. Components and Their Interactions

5.1 Component Responsibilities

Component	Responsibility
API Gateway	Single entry point for all clients; handles authentication, request routing, rate limiting, and load b
Baggage Tracking Service	Ingests RFID/IoT scan events, maintains the authoritative current location/status of each bag, a
Routing Optimisation Service	Consumes location-update events and airline operational data to compute or revise the optimal
AI Delay Prediction Service	Consumes tracking events and historical data to run a trained ML model that estimates delay pr
Notification Service	Subscribes to delay and status events and dispatches passenger-facing alerts via push notifica
Analytics & Reporting Service	Aggregates events and historical records into the analytics warehouse and produces operationa
Event Bus (Kafka/MQTT)	Provides asynchronous, durable publish-subscribe messaging that decouples producers and co
Data Layer	Polyglot persistence: a NoSQL store for high-write tracking data, a warehouse for analytics, a d

5.2 Component Interaction Workflow

The sequence below traces a representative flow: an RFID scan is captured, propagated through the event bus, evaluated by the AI prediction service, and — where a delay risk is detected — relayed to the passenger through the notification service.

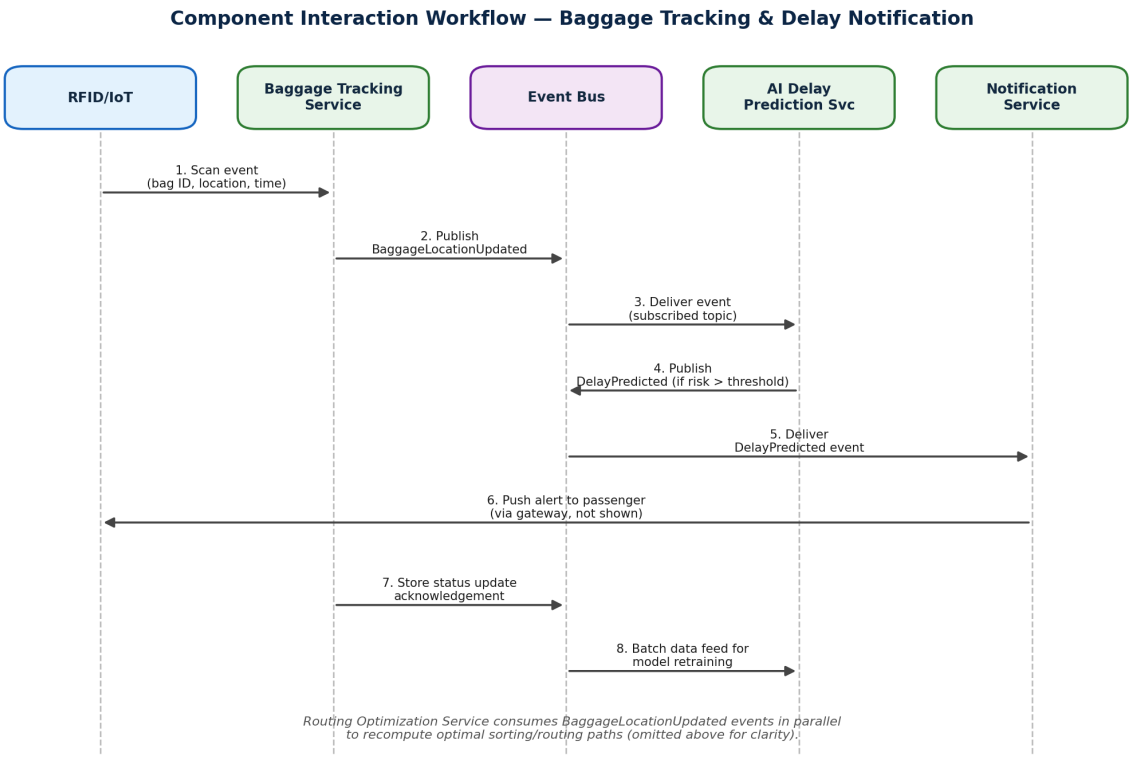

Figure 2: Component Interaction Workflow for a Baggage Scan and Delay Alert.

5.3 Data Flow Diagram

The Level-1 data flow diagram below shows how data moves between external entities, the six core processes, and the system's data stores, complementing the service-oriented view in Figure 1 with a process-and-data perspective.

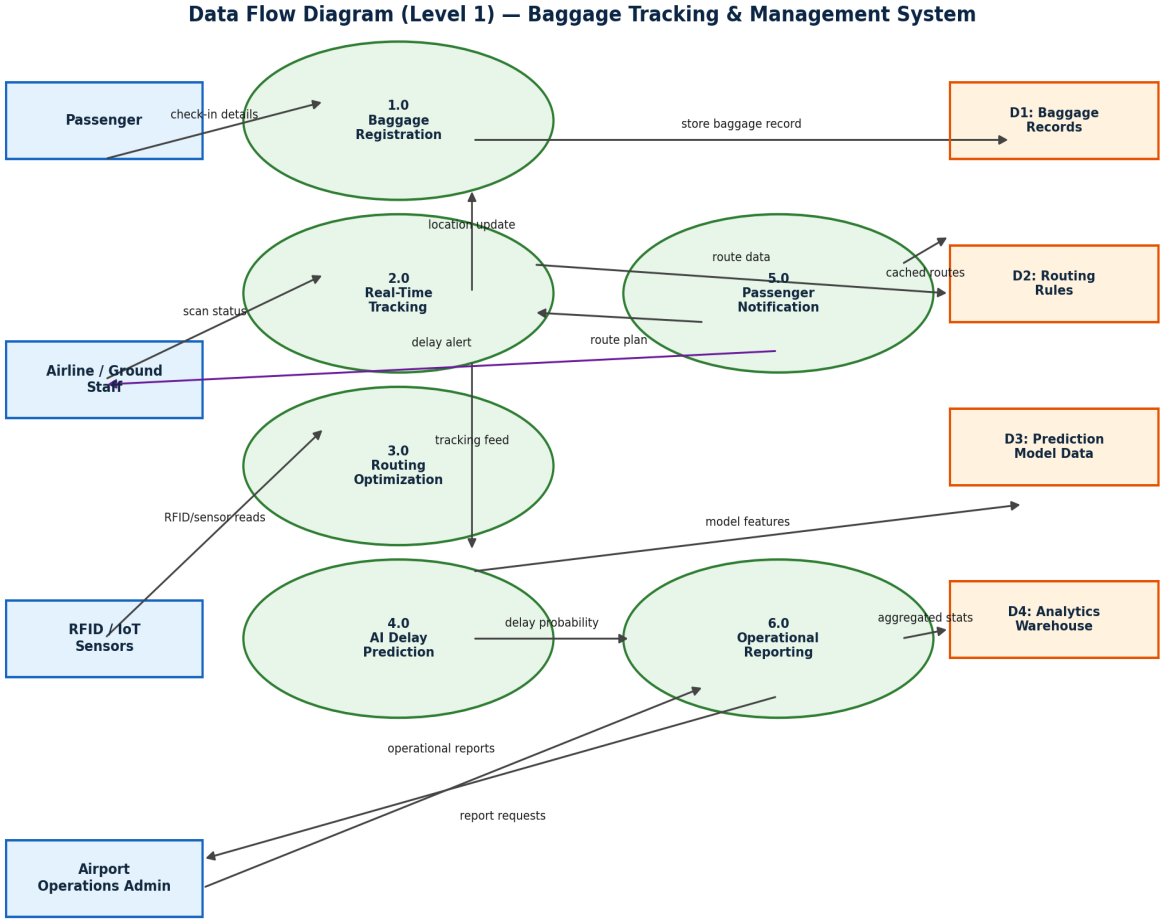

Figure 3: Data Flow Diagram (Level 1) for the Baggage Tracking and Management System.

6. Discussion of Quality Attributes

Attribute	How the Architecture Addresses It
Scalability	Each microservice scales independently and horizontally behind the API gateway; the event bus buffe
Security	The API gateway centralises authentication and authorisation; role-based access control separates pa
Performance	Asynchronous event delivery and a distributed cache keep read latency low; services are deployed clo
Reliability	The event bus provides durable, at-least-once delivery, so a scan event is not lost even if a downstrea
Maintainability	Clear service boundaries and independent deployability mean the AI model, a notification channel, or t
Availability	Redundant service instances behind the gateway, container orchestration (Kubernetes) for automatic r

7. Justification of Architectural Decisions

7.1 Rationale

The core architectural decision — separating tracking, routing, prediction, notification, and analytics into independent services connected via an event bus — follows directly from the requirement to process high-volume, bursty sensor data in near real time while allowing the AI prediction model to evolve independently of the rest of the platform. Splitting the AI component into its own service is a deliberate decision: model retraining, GPU/CPU scaling, and versioning needs differ substantially from the transactional needs of the tracking service, and coupling them would force unnecessary redeployment of stable tracking logic whenever the model changes.

7.2 Trade-offs Considered

- **Consistency vs. availability:** The event-driven design favours eventual consistency between services (e.g., the dashboard may lag the true bag location by a second or two) in exchange for higher availability and throughput; this trade-off is acceptable because exact real-time consistency is not safety-critical for this domain.
- **Operational complexity vs. flexibility:** Microservices and an event bus introduce more moving parts (service discovery, message broker, distributed tracing) than a monolithic design, but this complexity is justified by the independent scaling and deployment needs identified in the non-functional requirements.
- **Latency vs. decoupling:** Asynchronous messaging adds a small amount of latency compared to direct synchronous calls, but the resulting decoupling prevents a slow or failing downstream service (e.g., an SMS gateway) from blocking the core tracking pipeline.

7.3 Support for Future Enhancements and Long-Term Maintainability

- New capabilities (e.g., a computer-vision-based bag-damage detector, or a loyalty-tier prioritisation service) can be added as new microservices that simply subscribe to existing events on the bus, without modifying existing services.
- The AI Delay Prediction Service can be upgraded to newer model architectures or retrained on new data with no impact on the tracking or notification services, as long as the published event contract is preserved.
- The API Gateway provides a stable integration point for new airline or airport partner systems, insulating them from internal restructuring of the microservices layer.
- Containerised deployment (Kubernetes) supports incremental, service-by-service scaling as passenger and baggage volumes grow, without re-architecting the system.

8. Technology Stack Summary

The following technology choices support the architectural decisions discussed above and are representative of a production-grade implementation of this system.

Layer	Representative Technologies
Client / Presentation	React Native or Flutter (passenger app); React.js (staff dashboard and analytics console)
API Gateway	Kong, NGINX, or AWS API Gateway with OAuth2 / JWT-based authentication
Microservices	Spring Boot / Node.js (Express) / Python (FastAPI), packaged as Docker containers

AI / ML Component	Python with scikit-learn / TensorFlow for delay-prediction model training and inference
Event Bus	Apache Kafka (or MQTT for lightweight IoT telemetry)
Data Layer	MongoDB / Cassandra (tracking data), PostgreSQL or a data warehouse (analytics), Redis (cache),
Orchestration	Kubernetes for container orchestration, scaling, and rolling deployments
Notification Channels	Firebase Cloud Messaging (push), Twilio (SMS), SMTP-based email gateway

9. Conclusion

The Hybrid Microservices + Event-Driven Architecture selected for the Intelligent AI-Based Airport Baggage Tracking and Management System directly addresses the system's core requirements: real-time tracking under bursty sensor load, independent scalability of tracking, routing, prediction, and notification workloads, isolation of the AI/ML component for independent iteration, and resilience against partial failures. The architecture, its component interactions, and its data flows, as presented in Figures 1-3, provide a scalable, secure, and maintainable foundation that also accommodates future functional growth with minimal disruption to existing services.